

Sensor Demosaic v1.1

LogiCORE IP Product Guide

PG286 (v1.1) May 16, 2023

AMD Adaptive Computing is creating an environment where employees, customers, and partners feel welcome and included. To that end, we're removing non-inclusive language from our products and related collateral. We've launched an internal initiative to remove language that could exclude people or reinforce historical biases, including terms embedded in our software and IPs. You may still find examples of non-inclusive language in our older products as we work to make these changes and align with evolving industry standards. Follow this [link](#) for more information.



Table of Contents

Chapter 1: Introduction.....	4
Features.....	4
IP Facts.....	5
Chapter 2: Overview.....	6
Navigating Content by Design Process.....	8
Feature Summary.....	8
Applications.....	8
Licensing and Ordering.....	9
Chapter 3: Product Specification.....	10
Standards.....	10
Performance.....	10
Port Descriptions.....	11
Chapter 4: Designing with the Core.....	21
General Design Guidelines.....	21
Clocking.....	21
Resets.....	22
System Considerations.....	22
Chapter 5: Design Flow Steps.....	23
Customizing and Generating the Core.....	23
Constraining the Core.....	25
Simulation.....	26
Synthesis and Implementation.....	26
Chapter 6: Detailed Example Design.....	27
Synthesizable Example Design.....	28
Appendix A: Verification, Compliance, and Interoperability.....	31
Simulation.....	31



Hardware Testing.....	31
Interoperability.....	32
Appendix B: Upgrading.....	33
Appendix C: Debugging.....	34
Finding Help with AMD Adaptive Computing Solutions.....	34
Debug Tools.....	35
Hardware Debug.....	36
Appendix D: Additional Resources and Legal Notices.....	37
Support Resources.....	37
Finding Additional Documentation.....	37
References.....	38
Revision History.....	38
Please Read: Important Legal Notices.....	39

Introduction

The AMD LogiCORE™ IP Sensor Demosaic core provides an optimized hardware block that reconstructs sub-sampled color data for images captured by a Bayer image sensor. The color filter array overlaid on the silicon substrate enables CMOS or CCD image sensors to measure local light intensities that correspond to different wavelengths. However, the sensor measures the intensity of one principal color at any location. The Sensor Demosaic IP provides an efficient and low-footprint solution to interpolate the missing color components for every pixel.

Features

The Sensor Demosaic IP supports the following:

- RGB Bayer image sensor
- One, two, four, or eight pixel-wide AXI4-Stream video interface
- 8, 10, 12, and 16 bits per color component
- Spatial resolutions from 64 x 64 up to 8192 x 4320
- 8K60 fps in all supported device families

Note: Performance on low power devices can be sub-optimal.

IP Facts

AMD LogiCORE™ IP Facts Table	
Core Specifics	
Supported Device Family ¹	AMD Versal™ Adaptive SoC, AMD UltraScale+™ Families, 7 series
Supported User Interfaces	AXI4-Lite, AXI4-Stream
Resources	Performance and Resource Utilization web page
Provided with Core	
Design Files	Not Provided
Example Design	Yes
Test Bench	Not Provided
Constraints File	XDC
Simulation Model	Encrypted RTL, VHDL or Verilog Structural
Supported S/W Driver ³	Standalone, V4L2
Tested Design Flows ⁴	
Design Entry	AMD Vivado™ Design Suite
Simulation	For supported simulators, see the Vivado Design Suite User Guide: Release Notes, Installation and Licensing .
Synthesis	Vivado Synthesis
Support	
Release Notes and Known Issues	Master Answer Record: 68769
All Vivado IP Change Logs	Master Vivado IP Change Logs: 72775
Support web page	

Notes:

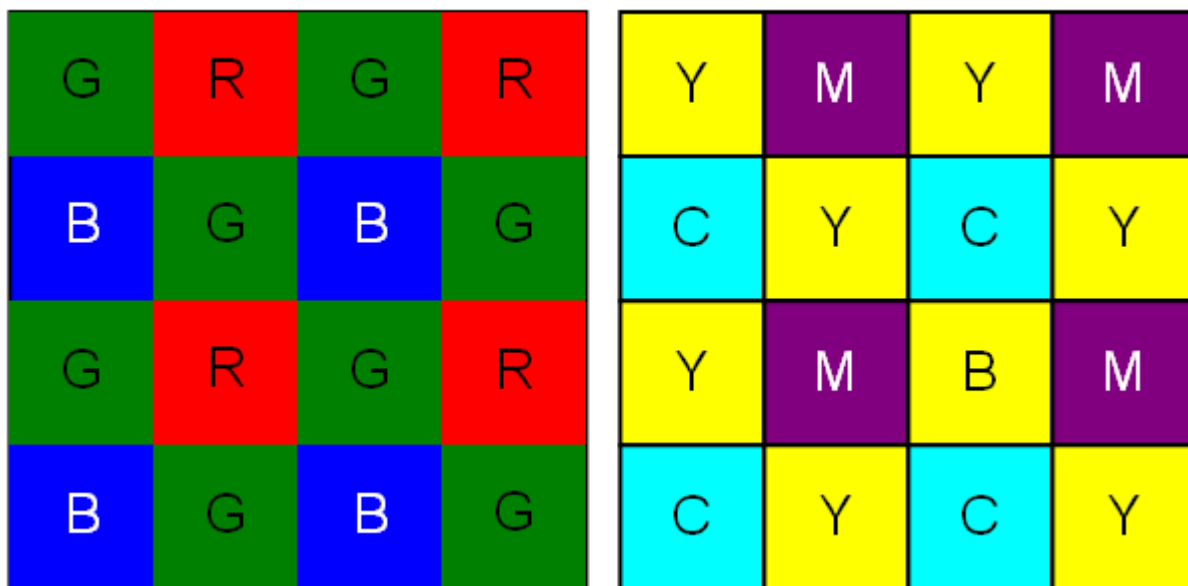
1. For a complete list of supported devices, see the AMD Vivado™ IP catalog.
2. Video protocol as defined in the Video IP: AXI Feature Adoption section of *Vivado Design Suite: AXI Reference Guide (UG1037)*.
3. Standalone driver details can be found in the Vitis directory: <install_directory>/Vitis/<release>/data/embeddedsw/doc/xilinx_drivers_api_toc.htm. [Linux Demosaic Driver Page](#).
4. For the supported versions of the tools, see the [Vivado Design Suite User Guide: Release Notes, Installation and Licensing](#).

Overview

Images captured by a CMOS/CCD image sensor are monochrome in nature. To generate a color image, three primary colors (typically red, green, and blue) are required for each pixel. Before the invention of color image sensors, the color image was created by superimposing three identical images with three different primary colors. These images were captured by placing different color filters in front of the sensor, allowing a certain bandwidth of the visible light to pass through.

Kodak scientist Dr. Bryce Bayer realized that an image sensor with a Color Filter Array (CFA) pattern would allow the reconstruction of all the colors of a scene from a single image capture. The color filter array is manufactured as part of the image sensor as a layer laid over the phototransistors. Example CFA patterns are shown in the following figure. These patterns are called Bayer patterns and are used in most digital imaging systems.

Figure 1: RGB and CMY Bayer Sensor Demosaic Patterns



X22003-112618

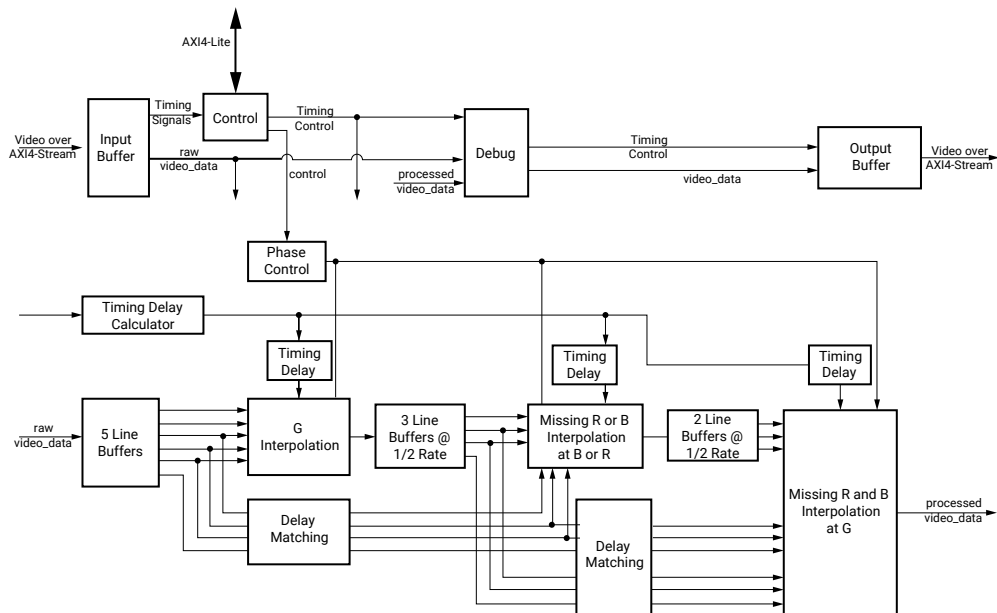
The original data for each pixel contains information only about one color, based on which color filter is positioned over that pixel. However, information for all three primary colors is needed at each pixel to reconstruct a color image. Some missing information can be recreated from the information available in neighboring pixels. This process of recreating the missing color information is called color interpolation or demosaicing, and might require dedicated hardware to process the image data in real-time.

There is no exact method to fully recover the missing information, as color channels have been physically sub-sampled by the CFA before proper low-pass filtering takes place, which might lead to aliasing between color channels.

Perfect recovery of the original signal might not be possible; however, the aliasing can be suppressed significantly by capitalizing on the temporal and spatial redundancies and structured nature of natural images/video sequences.

A variety of simple interpolation methods, such as Pixel Replication, Nearest Neighbor Interpolation, Bilinear Interpolation, and Bicubic Interpolation have been widely used for CFA demosaicing. Simple methods usually compromise quality, and more elaborate methods require the use of an external frame buffer. The Sensor Demosaic core was designed to efficiently suppress interpolation artifacts, such as the zipper and color aliasing effects, by minimizing Chrominance Variances in a 5x5 neighborhood, as illustrated in the following figure.

Figure 2: AMD Sensor Demosaic Block Diagram



X21999-112618

Image sensor data sheets specify whether the starting position, pixel(0,0) of the Bayer sampling grid is on a red-green or a blue-green line, and whether the first pixel is green. The Sensor Demosaic IP core supports all four possible Bayer phase combinations.

Navigating Content by Design Process

AMD Adaptive Computing documentation is organized around a set of standard design processes to help you find relevant content for your current development task. All AMD Versal™ adaptive SoC design process [Design Hubs](#) and the [Design Flow Assistant](#) materials can be found on the [Xilinx.com](#) website. This document covers the following design processes:

- **Hardware, IP, and Platform Development:** Creating the PL IP blocks for the hardware platform, creating PL kernels, functional simulation, and evaluating the AMD Vivado™ timing, resource use, and power closure. Also involves developing the hardware platform for system integration. Topics in this document that apply to this design process include:
 - [Port Descriptions](#)
 - [Register Space](#)
 - [Clocking](#)
 - [Resets](#)
 - [Customizing and Generating the Core](#)
 - [Chapter 6: Detailed Example Design](#)

Feature Summary

The Sensor Demosaic core reconstructs a color image from an RGB Bayer filtered sensor using a 5x5 interpolation aperture.

Applications

- Pre-processing block for image sensors
- Video surveillance
- Industrial imaging
- Video conferencing
- Machine vision

- Other imaging applications

Licensing and Ordering

This AMD LogiCORE™ IP module is provided at no additional cost with the AMD Vivado™ Design Suite under the terms of the [End User License](#).

The module is shipped as part of the AMD Vivado™ Design Suite. For full access to all core functionalities in simulation and in hardware, you must purchase a license for the core. To generate a full license, visit the [product licensing web page](#). Evaluation licenses and hardware timeout licenses might be available for this core. Contact your [local sales representative](#) for information about pricing and availability.

For more information about this core, visit the [Sensor Demosaic product web page](#).

Information about other AMD LogiCORE™ IP modules is available at the [Intellectual Property](#) page. For information about pricing and availability of other AMD LogiCORE IP modules and tools, contact your [local sales representative](#).

License Checkers

If the IP requires a license key, the key must be verified. The AMD Vivado™ design tools have several license checkpoints for gating licensed IP through the flow. If the license check succeeds, the IP can continue generation. Otherwise, generation halts with an error. License checkpoints are enforced by the following tools:

- Vivado Synthesis
- Vivado Implementation
- write_bitstream (Tcl command)



IMPORTANT! IP license level is ignored at checkpoints. The test confirms a valid license exists. It does not check IP license level.

Product Specification

Standards

The Sensor Demosaic core is compliant with the AXI4-Stream Video Protocol and AXI4-Lite interconnect standards. Refer to the Video IP: AXI Feature Adoption section of the *Vivado Design Suite: AXI Reference Guide* ([UG1037](#)) and AXI4-Stream Video IP and System Design Guide ([UG934](#)) for additional information.

Performance

The following sections detail the performance characteristics of the Sensor Demosaic core.

Maximum Frequencies

The following are typical clock frequencies for the target devices:

- AMD UltraScale+™ devices with speed grade of -1 or higher: 300 MHz
- AMD Virtex™ 7 and AMD Virtex™ UltraScale™ devices with speed grade of -2 or higher: 300 MHz
- AMD Kintex™ 7 and AMD Kintex™ UltraScale™ devices with speed grade of -2 or higher: 300 MHz
- AMD Artix™ 7 devices with speed grade of -2 or higher: 150 MHz

The maximum achievable clock frequency can vary. The maximum achievable clock frequency and all resource counts can be affected by other tool options, additional logic in the device, using a different version of AMD tools, and other factors.

Throughput

The Sensor Demosaic IP supports bi-directional data throttling between its AXI4-Stream slave and master interfaces. If the slave side data source is not providing valid data samples (`s_axis_video_tvalid` is not asserted), the core cannot produce valid output samples after its internal buffers are depleted. Similarly, if the master side interface is not ready to accept valid data samples (`m_axis_video_tready` is not asserted) the core cannot accept valid input samples after its buffers become full.

If the master interface is able to provide valid samples (`s_axis_video_tvalid` is High) and the slave interface is ready to accept valid samples (`m_axis_video_tready` is High), typically the core can process and produce one, two, four, or eight pixels specified by Samples Per Clock in the Vivado Integrated Design Environment (IDE) per `ap_clk` cycle.

However, at the end of each scan line and frame the core flushes internal pipelines for several clock cycles, during which the `s_axis_video_tready` is deasserted signaling that the core is not ready to process samples.

When the Sensor Demosaic IP is processing timed streaming video (which is typical for most video sources), the flushing periods coincide with the blanking periods and therefore do not reduce the throughput of the system.

When operating on a streaming video source (that is, not frame buffered data), the Sensor Demosaic must operate minimally at the burst data rate. For example, 148.5 MHz for a 1080p60 video source for a one sample per clock configuration of the IP. For a 4K 60 fps video source, the core must operate at 297 MHz for a two sample per clock configuration, or 148.5 MHz for a four sample per clock configuration on slower devices such as AMD Artix™ 7.

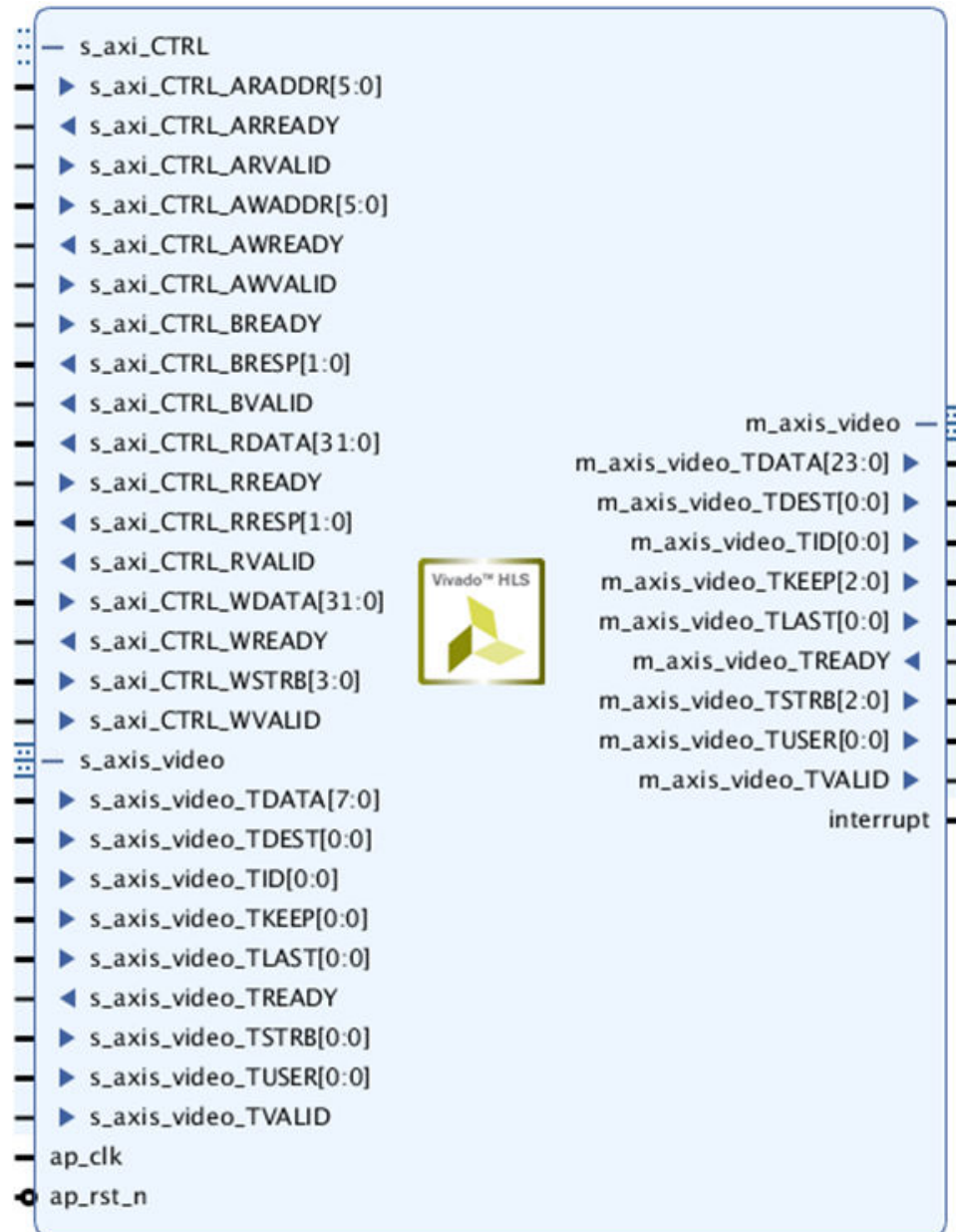
Resource Utilization

For full details about performance and resource utilization, visit the [Sensor Demosaic Performance and Resource Utilization web page](#).

Port Descriptions

The Sensor Demosaic core uses industry standard control and data interfaces to connect to other system components. The following sections describe the various interfaces available with the core. The following figure illustrates an I/O diagram of the Sensor Demosaic core.

Figure 3: Sensor Demosaic Core Top-Level Signaling Interface



Common Interface Signals

The following table summarizes the signals that are either shared by, or not part of the dedicated AXI4-Stream data or AXI4-Lite control interfaces.

Table 1: Common Interface Signals

Signal Name	Direction	Width	Description
AP_CLK	I	1	Video Core Clock
AP_RST_N	I	1	Video Core active-Low reset
INTERRUPT	O	1	Interrupt Request Pin

The AP_CLK and AP_RST_N signals are shared between the core, the AXI4-Stream data interfaces, and the AXI4-Lite control interface. The INTERRUPT pin is not supported and is reserved for future use.

AP_CLK

The AXI4-Stream and AXI4-Lite interfaces must be synchronous to the core clock signal AP_CLK. All AXI4-Stream interface input signals and AXI4-Lite control interface input signals are sampled on the rising edge of AP_CLK. All AXI4-Stream output signal changes occur after the rising edge of AP_CLK.

AP_RST_N

The AP_RST_N pin is an active-Low synchronous reset input pertaining to both AXI4-Lite and AXI4-Stream interfaces. When AP_RST_N is set to 0, the core resets at the next rising edge of AP_CLK.

Data Interface

The Sensor Demosaic core receives and transmits data using AXI4-Stream interfaces that implement a video protocol as defined in the Video IP: AXI Feature Adoption section of the (UG761) Vivado Design Suite: AXI Reference Guide (UG1037).

AXI4-Stream Signal Names and Descriptions

The following table describes the AXI4-Stream signal names and descriptions.



IMPORTANT! In the following table,

*TotalDataWidth=data_width*number_of_components*samples_per_clock*. The three values correspond to Maximum Data Width, Number of Video Components, and Samples Per Clock in the IDE respectively. Refer to [Chapter 5: Design Flow Steps](#) for more information.

Table 2: AXI4-Stream Data Interface Signal Descriptions

Signal Name	Direction	Width	Description
s_axis_video_tdata	I	$(data_width * samples_per_clock + 7) / 8 * 8$	Input Video Data

Table 2: AXI4-Stream Data Interface Signal Descriptions (cont'd)

Signal Name	Direction	Width	Description
s_axis_video_tvalid	I	1	Input Video Valid Signal
s_axis_video_tready	O	1	Input Ready
s_axis_video_tuser	I	1	Input Video Start Of Frame
s_axis_video_tlast	I	1	Input Video End Of Line
s_axi_video_tstrb	I	s_axis_video tdata/8	Input video data strobe indicates whether the content of the associated byte of TDATA is processed as a data byte or a position byte
s_axi_video_tkeep	I	s_axis_video tdata/8	Input video byte qualifier that indicates whether the content of the associated byte of TDATA is processed as part of the data stream
s_axi_video_tid	I	1	Input video data stream identifier
s_axi_video_tdest	I	1	Input video data routing information
m_axis_video_tdata	O	(TotalDataWidth+7)/8*8	Output Video Data
m_axis_video_tvalid	O	1	Output Valid
m_axis_video_tready	I	1	Output Ready
m_axis_video_tuser	O	1	Output Video Start Of Frame
m_axis_video_tlast	O	1	Output Video End Of Line
m_axi_video_tstrb	O	m_axis_video tdata/8	Output video data strobe indicates whether the content of the associated byte of TDATA is processed as a data byte or a position byte
m_axi_video_tkeep	O	m_axis_video tdata/8	Output video byte qualifier that indicates whether the content of the associated byte of TDATA is processed as part of the data stream
m_axi_video_tid	O	1	Output video data stream identifier
m_axi_video_tdest	O	1	Output video data routing information

Video Data

The AXI4-Stream interface specification restricts TDATA widths to integer multiples of 8 bits. Therefore, any bit data must be padded with zeros on the MSB to form a N*8 bit wide vector before connecting to s_axis_video_tdata. Padding does not affect the size of the core.

Similarly, data on the Sensor Demosaic output m_axis_video_tdata is packed and padded to multiples of 8 bits as necessary. Zero padding the most significant bits is only necessary for 10 and 12 bit wide data. The following tables explain the pixel mapping of AXI4-Stream interface with two pixels per clock and 10 bits per component configuration.

Table 3: Dual Pixels per Clock, 10 Bits per Component Mapping for RGB

63:60	59:50	49:40	39:30	29:20	19:10	9:0
zero padding	R1	B1	G1	R0	B0	G0

Table 4: Dual Pixels per Clock, 10 bits per Component Mapping for Bayer Sensor

23:20	19:10	9:0
Zero Padding	Bayer Sample 1	Bayer Sample 0

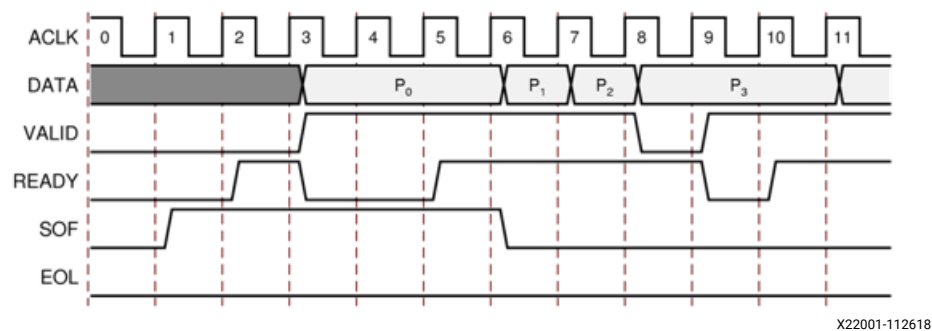
READY/VALID Handshake

A valid transfer occurs whenever `READY`, `VALID`, and `AP_RST_N` signals are High at the rising edge of `AP_CLK`, as seen in the following figure. During valid transfers, `DATA` only carries active video data. Blank periods and ancillary data packets are not transferred through the AXI4-Stream video protocol.

Guidelines on Driving `s_axis_video_tvalid`

When `s_axis_video_tvalid` is asserted, no interface signals (except the Sensor Demosaic core driving `s_axis_video_tready`) can change value until the transaction completes (`s_axis_video_tready`, `s_axis_video_tvalid`, and `AP_RST_N` are High on the rising edge of `AP_CLK`). When asserted, `s_axis_video_tvalid` can only be deasserted after a transaction has completed. Transactions cannot be retracted or aborted. In any cycle following a transaction, `s_axis_video_tvalid` can either be deasserted or remain asserted to initiate a new transfer.

Figure 4: Example of READY/VALID Handshake, Start of a New Frame



Guidelines on Driving `m_axis_video_tready`

The `m_axis_video_tready` signal can be asserted before, during, or after the cycle in which the Sensor Demosaic core asserts `m_axis_video_tvalid`. The assertion of `m_axis_video_tready` can be dependent on the value of `m_axis_video_tvalid`. A slave that can immediately accept data qualified by `m_axis_video_tvalid` should preassert its `m_axis_video_tready` signal until data is received. Alternatively, `m_axis_video_tready` can be registered and driven the cycle following `VALID` assertion.



RECOMMENDED: To minimize latency, your custom core slave interface should drive `READY` independently, or preassert `READY`.

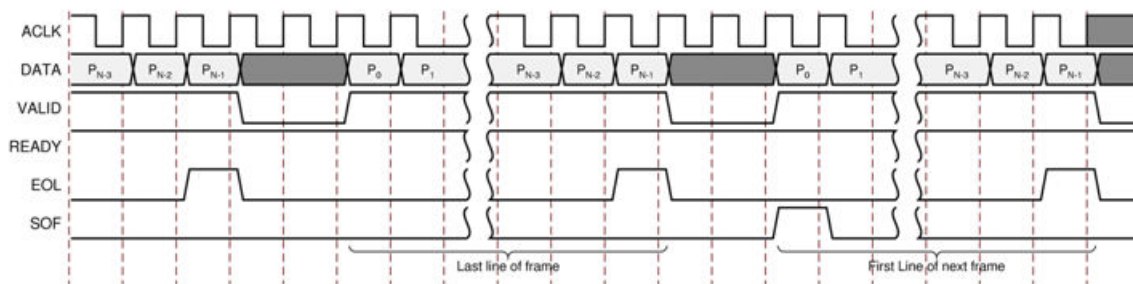
Start of Frame Signals: `m_axis_video_tuser0`, `s_axis_video_tuser0`

The Start-Of-Frame (SOF) signal, physically transmitted over the AXI4-Stream `TUSER0` signal, marks the first pixel of a video frame. The SOF pulse is one valid transaction wide, and must coincide with the first pixel of the frame, as seen in the previous figure. The SOF signal serves as a frame synchronization signal, which allows downstream cores to reinitialize, and detect the first pixel of a frame. The SOF signal can be asserted an arbitrary number of `AP_CLK` cycles before the first pixel value is presented on `DATA`, as long as a `VALID` is not asserted.

End of Line Signals: `m_axis_video_tlast`, `s_axis_video_tlast`

The End-Of-Line (EOL) signal, physically transmitted over the AXI4-Stream `TLAST` signal, marks the last pixel of a line. The EOL pulse is 1 valid transaction wide, and must coincide with the last pixel of a scanline, as seen in the following figure.

Figure 5: Use of EOL and SOF Signals



X22000-112618

Control Interface

The AXI4-Lite register interface dynamically controls the behavior of the core. The AXI4-Lite slave interface facilitates integrating the core into a processor system. Alternatively along with other video or AXI4-Lite compliant IP, the AXI4-Lite interface can be connected to an AXI4-Lite master. The core cannot be instantiated without the AXI4-Lite control interface.

AXI4-Lite Interface

The AXI4-Lite interface allows you to dynamically control parameters within the core. Core configuration can be accomplished using an AXI4-Lite master state machine, or an embedded Arm®, or a soft system processor such as a MicroBlaze™ processor.

The Sensor Demosaic core can be controlled through the AXI4-Lite interface by using functions provided by the driver in the Vitis software platform. Another method is performing read and write transactions to the Sensor Demosaic register space, but this method should only be used when the first method is not available.

Table 5: AXI4-Lite Interface Signals

Signal Name	Direction	Width	Description
s_axi_CTRL_awvalid	I	1	AXI4-Lite Write Address Channel Write Address Valid.
s_axi_CTRL_awread	O	1	AXI4-Lite Write Address Channel Write Address Ready. Indicates DMA ready to accept the write address.
s_axi_CTRL_awaddr	I	32	AXI4-Lite Write Address Bus.
s_axi_CTRL_wvalid	I	1	AXI4-Lite Write Data Channel Write Data Valid.
s_axi_CTRL_wready	O	1	AXI4-Lite Write Data Channel Write Data Ready. Indicates DMA is ready to accept the write data.
s_axi_CTRL_wdata	I	32	AXI4-Lite Write Data Bus.
s_axi_CTRL_bresp	O	2	AXI4-Lite Write Response Channel. Indicates results of the write transfer.
s_axi_CTRL_bvalid	O	1	AXI4-Lite Write Response Channel Response Valid. Indicates response is valid.
s_axi_CTRL_bready	I	1	AXI4-Lite Write Response Channel Ready. Indicates target is ready to receive response.
s_axi_CTRL_arvalid	I	1	AXI4-Lite Read Address Channel Read Address Valid.
s_axi_CTRL_arready	O	1	Ready. Indicates DMA is ready to accept the read address.
s_axi_CTRL_araddr	I	32	AXI4-Lite Read Address Bus.
s_axi_CTRL_rvalid	O	1	AXI4-Lite Read Data Channel Read Data Valid.

Table 5: AXI4-Lite Interface Signals (cont'd)

Signal Name	Direction	Width	Description
s_axi_CTRL_rready	I	1	AXI4-Lite Read Data Channel Read Data Ready. Indicates target is ready to accept the read data.
s_axi_CTRL_rdata	O	32	AXI4-Lite Read Data Bus
s_axi_CTRL_rresp	O	2	AXI4-Lite Read Response Channel Response. Indicates results of the read transfer.

Register Space

The core has seven core-specific registers that allow dynamic control of the core's operation. All registers have an initial value of 0. The following table describes the register names.

Table 6: Register Names and Descriptions

Address (hex) BASEADDR +	Register Name	Access Type	Register Description
0x0000	Control	R/W	Bit 0: ap_start (R/W/COH) ¹ Bit 1: ap_done (R/COR) ¹ Bit 2: ap_idle (R) Bit 3: ap_ready (R) Bit 7: auto_restart (R/W) Others: reserved
0x0004	Global Interrupt Enable	R/W	Bit 0: Global Interrupt Enable Others: reserved This register is not used but reserved for future use.
0x0008	IP Interrupt Enable Register	R/W	Bit 0: ap_done Bit 1: ap_ready Others: reserved This register is not used but reserved for future use.
0x000C	IP Interrupt Status Register	R/TOW ¹	Bit 0: ap_done Bit 1: ap_ready Others: reserved This register is not used but reserved for future use.
0x0010	Active Width	R/W	Number of Active Pixels per Scanline
0x0018	Active Height	R/W	Number of Active Lines per Frame

Table 6: Register Names and Descriptions (cont'd)

Address (hex) BASEADDR +	Register Name	Access Type	Register Description
0x0028	Bayer Phase	R/W	Bits 1-0: Bayer sampling grid starting position

Notes:

1. COR = Clear on Read, COH - Clear on Handshake, TOW- Toggle on Write
2. Control Register (0x0000), Global Interrupt Enable Register (0x0004), IP Interrupt Enable Register (0x0008), and IP Interrupt Status Register (0x000C) are explained in "S_AXILITE Control Register Map" of the *Vitis High-Level Synthesis User Guide* (UG1399). The definitions of these registers in UG1399 can have some additional bits; however, in this IP, we are accessing only bits mentioned in the above table. Therefore, only the bits in the table need to be considered while accessing the Control Register, Global Interrupt Enable Register, IP Interrupt Enable Register, and IP Interrupt Status Register.

CONTROL (0x0000) Register

The Control register controls the operation of the core. Bit[0] of the Control register, `ap_start`, initiates the core from software. Writing 1 to this bit starts the core to generate a video frame.

Bit[1] of the Control register, `ap_done`, indicates when the IP has completed all operations in the current transaction. A logic 1 on this signal indicates that the IP has completed all operations in this transaction.

Bit[2] of the Control register, `ap_idle`, indicates if the IP is operating or idle (no operation). The idle state is indicated by logic 1. This signal is asserted low once the IP starts operating. This signal is asserted high when the IP completes operation and no further operations are performed.

Bit[3] of the Control register, `ap_ready`, indicates when the IP is ready for new inputs. It is set to logic 1 when the IP is ready to accept new inputs, indicating that all input reads for this transaction are completed. If the IP has no operations in the pipeline, new reads are not performed until the next transaction starts. This signal is used to make a decision on when to apply new values to the input ports and whether to start a new transaction.

Bit[7] of the Control register, `auto_restart`, can be set to enable the auto-restart mode. Thereafter, the IP restarts automatically at the end of each transaction.

ACTIVE_WIDTH (0x0010) Register

The `active_width` register encodes the number of active pixels per scan. Supported values are 64 and the value provided in the Maximum number of Columns field in the IDE. To avoid processing errors, you should restrict values written to `active_width` to the range supported by the core instance.

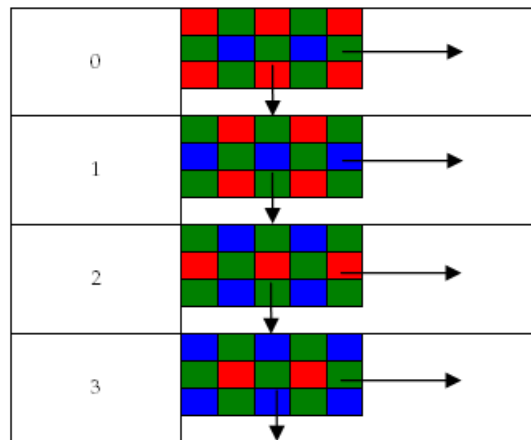
ACTIVE_HEIGHT (0x0018) Register

The `active_height` register encodes the number of active scan lines per frame. Supported values are between 64 and the value provided in the Maximum number of Rows field in the IDE. To avoid processing errors, you should restrict values written to `active_height` to the range supported by the core instance.

BAYER_PHASE (0x0028) Register

Image sensor data sheets specify whether the starting position, pixel(0,0), of the Bayer sampling grid is on a red-green or blue-green line, and whether the first pixel is green or not. The AMD Sensor Demosaic IP supports all four possible Bayer phase combinations. Bits 1:0 specify whether the top-left corner of the Bayer sampling grid starts with Green, Red, or Blue Pixel. The figure below displays the top-left corner of the image sample matrix along with the Bayer Phase Register value combinations.

Figure 6: Bayer Phase Register Combination Definitions



X22002-112618

Designing with the Core

General Design Guidelines

The Sensor Demosaic core interpolates Bayer sub-sampled image sensor data to downstream processing modules that process RGB data. The samples for core processes are provided using an AXI4-Lite slave interface. The output pixels are provided using an AXI4-Lite master interface and are controlled using an AXI4-Lite interface. The Sensor Demosaic block cannot change the input/output image sizes, the input and output pixel clock rates, or the frame rate.



RECOMMENDED: *The Sensor Demosaic core is designed to be used with the Video In to AXI4-Lite and Video Timing Controller cores.*

The Video Timing Controller core measures the timing parameters, such as the number of active scan lines and the number of active pixels per scan line of the image sensor. The Video In to AXI4-Stream core formats couples the sensor data interface to AXI4-Stream.

Clocking

The Sensor Demosaic has only one clock domain. All interfaces (master and slave AXI4-Stream video interfaces and AXI4-Lite interface) use the `ap_clk` pin as its clock source.

The pixel throughput of the Sensor Demosaic core is defined by the product of the clock frequency times the Samples per Clock setting in the AMD Vivado™ IDE. With a clock frequency of 300 MHz for `ap_clk`, and a two sample per clock configuration, the Sensor Demosaic is capable of a 600 mega pixel throughput rate, which is sufficient to handle 4K resolutions at 60Hz.

Resets

The Sensor Demosaic only has a hardware reset option: the `ap_rst_n` pin. No software reset option is available. The external reset pulse needs to be held for 16 or more `ap_clk` cycles to reset the core. The `ap_rst_n` signal is synchronous to the `ap_clk` clock domain. The `ap_rst_n` signal resets the entire core, including the AXI4-Lite and AXI4-Stream interfaces.

System Considerations

The Sensor Demosaic core must be configured for the actual input image frame size to operate properly. To gather the frame size information from the image video stream, it can be connected to the Video In to AXI4-Stream input and the Video Timing Controller. The timing detector logic in the Video Timing Controller gathers the video timing signals. The AXI4-Lite control interface on the Video Timing Controller allows the system processor to read out the measured frame dimensions, and program all downstream cores, such as the Sensor Demosaic, with the appropriate image dimensions.

Programming Sequence

All Sensor Demosaic processing parameters other than image size can be changed dynamically on a frame-by-frame basis; the change is picked up immediately. If the image size needs to be changed on the fly or the entire system needs to be restarted, it is recommended that pipelined AMD IP video cores are disabled or reset from system output towards the system input, and programmed or enabled from system output to system input.

Design Flow Steps

This section describes customizing and generating the core, constraining the core, and the simulation, synthesis, and implementation steps that are specific to this IP core. More detailed information about the standard AMD Vivado™ design flows and the IP integrator can be found in the following Vivado Design Suite user guides:

- *Vivado Design Suite User Guide: Designing IP Subsystems using IP Integrator* ([UG994](#))
- *Vivado Design Suite User Guide: Designing with IP* ([UG896](#))
- *Vivado Design Suite User Guide: Getting Started* ([UG910](#))
- *Vivado Design Suite User Guide: Logic Simulation* ([UG900](#))

Customizing and Generating the Core

This section includes information about using AMD tools to customize and generate the core in the AMD Vivado™ Design Suite.

If you are customizing and generating the core in the Vivado IP integrator, see the *Vivado Design Suite User Guide: Designing IP Subsystems using IP Integrator* ([UG994](#)) for detailed information. IP integrator might auto-compute certain configuration values when validating or generating the design. To check whether the values do change, see the description of the parameter in this chapter. To view the parameter value, run the `validate_bd_design` command in the Tcl console.

You can customize the IP for use in your design by specifying values for the various parameters associated with the IP core using the following steps:

1. Select the IP from the IP catalog.
2. Double-click the selected IP or select the Customize IP command from the toolbar or right-click menu.

For details, see the *Vivado Design Suite User Guide: Designing with IP* ([UG896](#)) and the *Vivado Design Suite User Guide: Getting Started* ([UG910](#)).

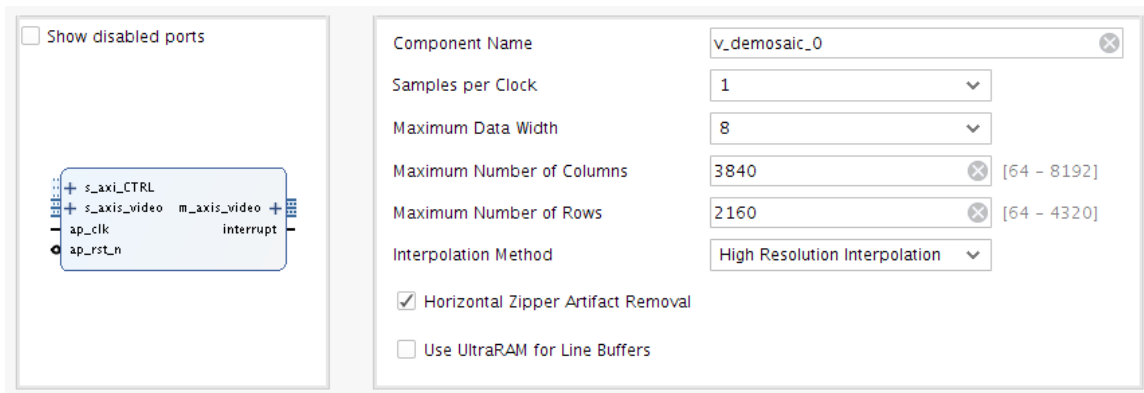
Note:

Figures in this chapter are illustrations of the Vivado IDE. The layout depicted here might vary from the current version.

Interface

The AMD Sensor Demosaic core is easily configured to meet your specific requirements through the Vivado Design Suite. This section provides a quick reference to parameters that can be configured at generation time.

Figure 7: Customize IP Screen



The screen displays a representation of the IP symbol on the left side, and the parameter assignments on the right side, which are described as follows:

- **Component Name:** The component name is used as the base name of output files generated for the module. Names must begin with a letter and must be composed of the following characters: a to z, 0 to 9, and “_”.
- **Samples Per Clock:** Specifies the number of pixel processed per clock cycle. Permitted values are 1, 2, 4, and 8 samples per clock. This parameter determines the throughput of the IP. The more samples per clock, the larger throughput it provides. The larger throughput always needs more hardware resources.
- **Maximum Data Width:** Specifies the bit width of input samples. Permitted values are 8, 10, 12, and 16 bits. This parameter should match the Video Component Width of the video IP core connected to the slave AXI4-Stream video interface.
- **Maximum Number of Columns:** Specifies maximum video columns/pixels the IP core could produce at runtime. Any video width that is less than the Maximum Number of Columns can be programmed through AXI4-Lite control interface without regenerating the core.
- **Maximum Number of Rows:** Specifies maximum video rows/lines the IP core could produce at runtime. Any video height that is less than the Maximum Number of Rows can be programmed through AXI4-Lite control interface without regenerating the core.
- **Interpolation Method:** There are two interpolation methods to choose from:

- **Fringe Tolerant Interpolation:** This method produces softer images with suppressed fringing artifacts. Use this option for low-cost optics that could possibly introduce color fringing artifacts.
- **High Resolution Interpolation:** This method is suggested for high quality optics and applications where high resolution is essential. Selecting this method uses more block RAMs and slices, and approximately doubles the number of DSP48s.
- **Horizontal Zipper Artifact Removal:** This option adds a post processing smoothing filter to remove horizontal zipper artifacts. The post processing filter softens the output image, and incurs some additional slice resources.
- **Use UltraRAM for Line Buffers:** In UltraScale+ devices, line buffers used for interpolating the pixels can be stored in UltraRAM instead of Block RAM.

User Parameters

The following table shows the relationship between the fields in the Vivado IDE and the User Parameters (which can be viewed in the Tcl Console).

Table 7: Vivado IDE Parameter to User Parameter Relationship

Vivado IDE Parameter/Value	User Parameter/Value	Default Value
Top-Level Parameters		
Samples per Clock	SAMPLES_PER_CLOCK	1
Maximum Number of Columns	MAX_COLS	3840
Maximum Number of Rows	MAX_ROWS	2160
Maximum Data Width	MAX_DATA_WIDTH	8
Interpolation Method	ALGORITHM	0
High Resolution Interpolation	0	
Fringe Tolerant Interpolation	1	
Horizontal Zipper Artifact Removal	ENABLE_ZIPPER_REMOVAL	true
Use UltraRAM for Line Buffers	USE_URAM	0

Output Generation

For details, see “Generating IP Output Products” in the *Vivado Design Suite User Guide: Designing with IP* ([UG896](#)).

Constraining the Core

This section contains information about constraining the core in the Vivado Design Suite.

Required Constraints

The only constraints required are clock frequency constraints for the core clock, `ap_clk`. Paths from AXI4-Lite signals should be constrained with a `set_false_path`, causing setup and hold checks to be ignored for AXI4-Lite signals. These constraints are provided in the XDC constraints file included with the core.

Simulation

For comprehensive information about AMD Vivado™ simulation components, as well as information about using supported third-party tools, see the *Vivado Design Suite User Guide: Logic Simulation* ([UG900](#)).

Synthesis and Implementation

For details about synthesis and implementation, see the *Vivado Design Suite User Guide: Designing with IP* ([UG896](#)).

Detailed Example Design

This chapter provides two example systems that include the Sensor Demosaic core, a simulation example design and a synthesizable example design. The example designs highlight the following key system-level aspects of designing with the Sensor Demosaic core:

- Typical usage of sensor demosaic with other cores and AXI master.
- Configuration of sensor demosaic registers on the fly.

The supported platforms are listed in the following table.

Table 8: Supported Platforms

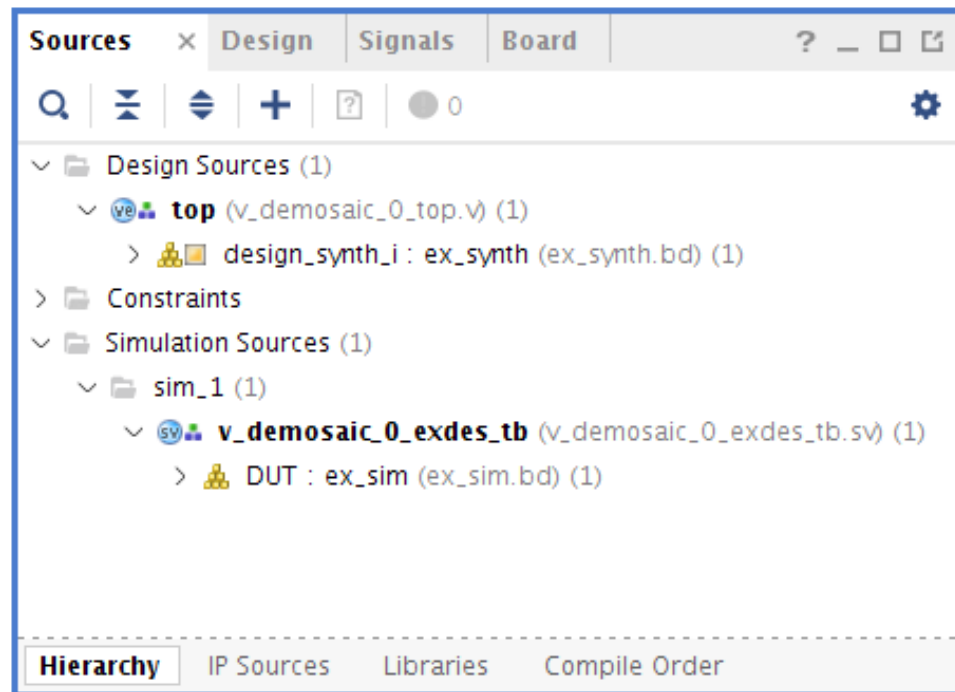
Development Boards	Additional Hardware	Processor
KC705	N/A	MicroBlaze™ processor
ZCU102	N/A	A53
ZCU104	N/A	A53
ZCU106	N/A	A53
VCK190	N/A	CIPS

To open the example project, perform the following steps:

1. Select the **Sensor Demosaic IP** from IP Catalog.
2. Double-click on the selected IP or right-click the IP and select **Customize IP** from the menu.
3. Configure the build-time parameters in the Customize IP window and click OK. The AMD Vivado™ IDE generates an example design matching the build-time configuration.
4. In the Generate Output Products window, select **Generate** or **Skip**. If Generate is selected, the output products of the IP are generated after a brief moment.
5. Right-click **Sensor Demosaic** in Sources panel and select **Open IP Example Design** from the menu.
6. In the Open IP Example Design window, select example project directory and click **OK**. The Vivado software then runs automation to generate example design in selected directory.

The generated project contains two example designs. The following figure shows the Source panel of the example project. The synthesizable example block design, along with the top-level file, resides in the Design Sources catalog. Simulation example design files (including the block design file, SystemVerilog test bench, and another task file) are under Simulation Sources.

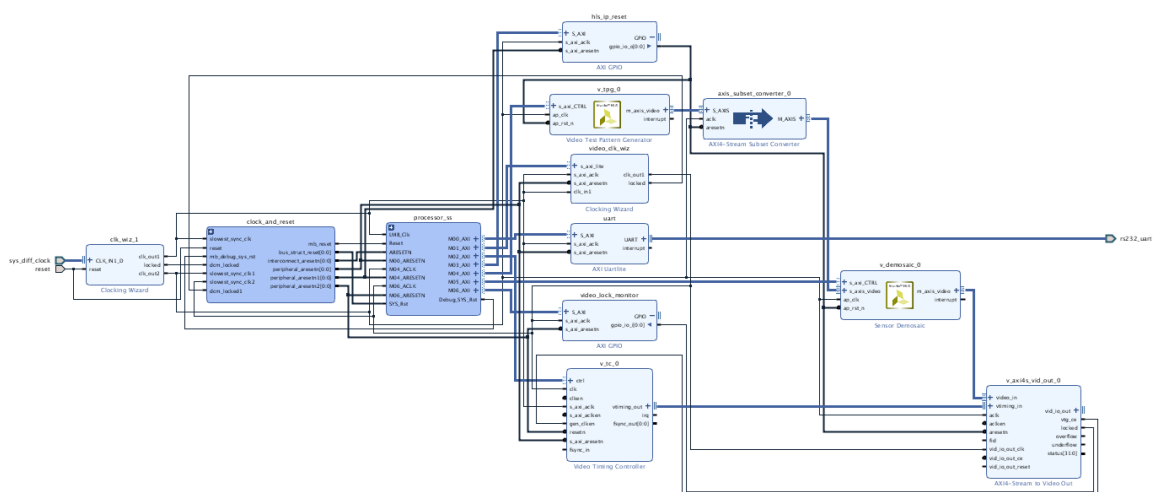
Figure 8: Example Project Source Panel



Synthesizable Example Design

The following synthesizable example design requires both AMD Vivado™ and the AMD Vitis™ software platform.

Figure 9: Synthesizable Example Block Design



The difference between the synthesizable design and the simulation example design is the use of a processor instead of the AXI VIP core as AXI master. The locked port of AXI4-Stream to Video Out is connected to the axi_gpio_lock core and the processor polls the corresponding register for a sign that the test passed.

The first step is to run synthesis, implementation, and bitstream generation in Vivado. After all those steps are done, select **File → Export → Export Hardware**. In the window, select **Include bitstream**, select an export directory and click **OK**.

The remaining work is performed in the Vitis software platform. The Sensor Demosaic example design file can be found at Vitis software platform directory:

```
<install_directory>/<release>/data/embeddedsw/XilinxProcessorIPLib/
drivers/v_demosaic_v2_0/examples/
```

Example application design source files (contained within the *examples* folder) are tightly coupled with the Sensor Demosaic example design available in AMD Vivado™ Catalog.

The `vdemosaic_example.tcl` file automates the process of generating the downloadable bit and `.elf` files from the provided example `.hdf` file.

To run the provided Tcl script:

1. Copy the exported example design hdf file in the *examples* directory of the driver.
2. Launch the AMD Software Command-Line Tool (`xsc`) terminal.
3. Use `cd` to access the *examples* directory.
4. Source the `.tcl` file:

```
xsc>source vdemosaic_example.tcl
```

5. Execute the script:

```
xsc>vdemosaic_example <xsa_file_name.xsa>
```

The Tcl script performs the following actions:

- Creates the workspace
- Creates the HW project
- Creates BSP
- Creates application project
- Builds BSP and application project

After the process is complete, the required files are available in:

```
bit file -> vdemosaic_example_hw_platform/hw folder
```

```
elf file -> vdemosaic_example.sdk/vtpg_example_design/{Debug/Release}  
folder
```

Next, perform the following steps to run the software application:



IMPORTANT! *To do so, make sure that the hardware is powered on and a Digilent Cable or an USB Platform Cable is connected to the host PC. Also, ensure that a USB cable is connected to the UART port of the board.*

1. Launch the Vitis software platform.
2. Set the workspace to the `vdemosaic_example.sdk` folder in the prompted window. The Vitis software platform project opens automatically. If a welcome page shows up, close that page.
3. Download the bitstream into the FPGA by selecting **Xilinx Tools > Program FPGA**. The Program FPGA dialog box opens.
4. Ensure that the Bitstream field shows the bitstream file generated by the Tcl script, and then click **Program**.
Note: The DONE LED on the board turns green if the programming is successful.
5. A terminal program (Hyper Terminal or PuTTY) is needed for UART communication. Open the program, choose the appropriate port, set the baud rate to 115200, and establish a serial port connection.
6. Select and right-click the `vdemosaic_example_design` application in the Project_Explorer panel.
7. Select **Run As > Launch** on Hardware (System Debugger).
8. Select the **Binaries** and **Qualifier** in the window, and click **OK**. The example design test results are shown in the terminal program.

For more information, visit <https://www.xilinx.com/products/design-tools/vitis.html>.

When executed on the board, the example application performs the following actions:

- Programs the Video Clock Generator to 1080p@60Hz
- Programs the TPG and Sensor Demosaic to 1080p@60Hz
- Checks for Video Lock and reports the status (PASS/FAIL) on UART
- Repeats steps 1 to 3 for 4KP@30Hz and 4KP@60Hz

Verification, Compliance, and Interoperability

Simulation

A highly parameterizable test bench was used to test the Sensor Demosaic core in AMD Vitis™ HLS. Testing included the following:

- Register accesses
 - Processing multiple frames of data
 - Varying IP throughput and pixel data width
 - Testing of various frame sizes
 - Varying parameter settings
-

Hardware Testing

The Sensor Demosaic core has been validated in hardware by AMD to represent a variety of parameterizations, including the following:

- A test design was developed for the core that incorporated a processor, AXI4-Lite interconnect and various other peripherals. The processor was responsible for the following:
 - Programming the video clock to match tested video resolution
 - Configuring the Sensor Demosaic core with different resolutions
 - Launching the test
 - Reporting the Pass/Fail status of the test and any errors that were found

Interoperability

The core slave (input) AXI4-Stream interface can work directly with Bayer pattern data, and the core produces RGB video data.

Upgrading

Video Demosaic v1.1 is a direct replacement for v1.0. The IP features remain the same; however, AMD Vitis™ HLS is used to synthesize the IP instead of AMD Vivado™ HLS.

Debugging

This appendix includes details about resources available on the AMD Support website and debugging tools.

If the IP requires a license key, the key must be verified. The AMD Vivado™ design tools have several license checkpoints for gating licensed IP through the flow. If the license check succeeds, the IP can continue generation. Otherwise, generation halts with an error. License checkpoints are enforced by the following tools:

- Vivado Synthesis
- Vivado Implementation
- write_bitstream (Tcl command)



IMPORTANT! IP license level is ignored at checkpoints. The test confirms a valid license exists. It does not check IP license level.

Finding Help with AMD Adaptive Computing Solutions

To help in the design and debug process when using the core, the [Support web page](#) contains key resources such as product documentation, release notes, answer records, information about known issues, and links for obtaining further product support. The [Community Forums](#) are also available where members can learn, participate, share, and ask questions about AMD Adaptive Computing solutions.

Documentation

This product guide is the main document associated with the core. This guide, along with documentation related to all products that aid in the design process, can be found on the [Support web page](#) or by using the AMD Adaptive Computing Documentation Navigator. Download the Documentation Navigator from the [Downloads page](#). For more information about this tool and the features available, open the online help after installation.

Answer Records

Answer Records include information about commonly encountered problems, helpful information on how to resolve these problems, and any known issues with an AMD Adaptive Computing product. Answer Records are created and maintained daily to ensure that users have access to the most accurate information available.

Answer Records for this core can be located by using the Search Support box on the main [Support web page](#). To maximize your search results, use keywords such as:

- Product name
- Tool message(s)
- Summary of the issue encountered

A filter search is available after results are returned to further target the results.

Master Answer Record for the Core

AR: [68769](#).

Technical Support

AMD Adaptive Computing provides technical support on the [Community Forums](#) for this AMD LogiCORE™ IP product when used as described in the product documentation. AMD Adaptive Computing cannot guarantee timing, functionality, or support if you do any of the following:

- Implement the solution in devices that are not defined in the documentation.
- Customize the solution beyond that allowed in the product documentation.
- Change any section of the design labeled DO NOT MODIFY.

To ask questions, navigate to the [Community Forums](#).

Debug Tools

There are many tools available to address Sensor Demosaic IP design issues. It is important to know which tools are useful for debugging various situations.

Vivado Design Suite Debug Feature

The AMD Vivado™ Design Suite debug feature inserts logic analyzer and virtual I/O cores directly into your design. The debug feature also allows you to set trigger conditions to capture application and integrated block port signals in hardware. Captured signals can then be analyzed. This feature in the Vivado IDE is used for logic debugging and validation of a design running in AMD devices.

The Vivado logic analyzer is used to interact with the logic debug LogiCORE IP cores, including:

- ILA 2.0 (and later versions)
- VIO 2.0 (and later versions)

See the *Vivado Design Suite User Guide: Programming and Debugging* ([UG908](#)).

Hardware Debug

Hardware issues can range from link bring-up to problems seen after hours of testing. This section provides debug steps for common issues. The AMD Vivado™ debug feature is a valuable resource to use in hardware debug. The signal names mentioned in the following individual sections can be probed using the debug feature for debugging the specific problems.

General Checks

Ensure that all the timing constraints for the core were properly incorporated from the example design and that all constraints were met during implementation.

- Does it work in post-place and route timing simulation? If problems are seen in hardware but not in timing simulation, this could indicate a PCB issue. Ensure that all clock sources are active and clean.
- If using MMCMs in the design, ensure that all MMCMs have obtained lock by monitoring the `locked` port.

Additional Resources and Legal Notices

Support Resources

For support resources such as Answers, Documentation, Downloads, and Forums, see [Support](#).

Finding Additional Documentation

Documentation Portal

The AMD Adaptive Computing Documentation Portal is an online tool that provides robust search and navigation for documentation using your web browser. To access the Documentation Portal, go to <https://docs.xilinx.com>.

Documentation Navigator

Documentation Navigator (DocNav) is an installed tool that provides access to AMD Adaptive Computing documents, videos, and support resources, which you can filter and search to find information. To open DocNav:

- From the AMD Vivado™ IDE, select **Help** → **Documentation and Tutorials**.
- On Windows, click the **Start** button and select **Xilinx Design Tools** → **DocNav**.
- At the Linux command prompt, enter `docnav`.

Design Hubs

AMD Design Hubs provide links to documentation organized by design tasks and other topics, which you can use to learn key concepts and address frequently asked questions. To access the Design Hubs:

- In DocNav, click the **Design Hubs View** tab.

- Go to the [Design Hubs](#) webpage.

Note: For more information on DocNav, see the [Documentation Navigator](#) webpage.

References

These documents provide supplemental material useful with this guide:

1. *Vivado Design Suite: AXI Reference Guide* ([UG1037](#))
2. *AXI4-Stream Video IP and System Design Guide* ([UG934](#))
3. *ISE to Vivado Design Suite Migration Guide* ([UG911](#))
4. *Vivado Design Suite User Guide: Designing with IP* ([UG896](#))
5. *Vivado Design Suite User Guide: Programming and Debugging* ([UG908](#))
6. *Vivado Design Suite User Guide: Getting Started* ([UG910](#))
7. *Vivado Design Suite User Guide: Logic Simulation* ([UG900](#))
8. *Vivado Design Suite User Guide: Designing IP Subsystems using IP Integrator* ([UG994](#))
9. *Vitis High-Level Synthesis User Guide* ([UG1399](#))

Revision History

The following table shows the revision history for this document.

Section	Revision Summary
05/16/2023 Version 1.1	
IP Facts	Updated Resources URL in IP Facts table.
05/11/2022 Version 1.1	
Register Space	Updated the register description.
08/09/2021 Version 1.1	
General updates	Added support for AMD Versal™ example design in Table 8: Supported Platforms
02/04/2021 Version 1.1	
General updates	Updated to support version 1.1
11/15/2019 Version 1.0	
Synthesizable Example Design	Updated with the Vitis software platform flow.
12/05/2018 Version 1.0	
Register Space	Updated Bayer Phase register address from 0x0020 to 0x0028

Section	Revision Summary
04/04/2018 Version 1.0	
General updates	Added option to use UltraRAM for line buffers on UltraScale+ devices. Added support for ZCU102, ZCU104, and ZCU106 boards in the synthesizable example design.
10/04/2017 Version 1.0	
General updates	Initial AMD release.

Please Read: Important Legal Notices

The information presented in this document is for informational purposes only and may contain technical inaccuracies, omissions, and typographical errors. The information contained herein is subject to change and may be rendered inaccurate for many reasons, including but not limited to product and roadmap changes, component and motherboard version changes, new model and/or product releases, product differences between differing manufacturers, software changes, BIOS flashes, firmware upgrades, or the like. Any computer system has risks of security vulnerabilities that cannot be completely prevented or mitigated. AMD assumes no obligation to update or otherwise correct or revise this information. However, AMD reserves the right to revise this information and to make changes from time to time to the content hereof without obligation of AMD to notify any person of such revisions or changes. THIS INFORMATION IS PROVIDED "AS IS." AMD MAKES NO REPRESENTATIONS OR WARRANTIES WITH RESPECT TO THE CONTENTS HEREOF AND ASSUMES NO RESPONSIBILITY FOR ANY INACCURACIES, ERRORS, OR OMISSIONS THAT MAY APPEAR IN THIS INFORMATION. AMD SPECIFICALLY DISCLAIMS ANY IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, OR FITNESS FOR ANY PARTICULAR PURPOSE. IN NO EVENT WILL AMD BE LIABLE TO ANY PERSON FOR ANY RELIANCE, DIRECT, INDIRECT, SPECIAL, OR OTHER CONSEQUENTIAL DAMAGES ARISING FROM THE USE OF ANY INFORMATION CONTAINED HEREIN, EVEN IF AMD IS EXPRESSLY ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

AUTOMOTIVE APPLICATIONS DISCLAIMER

AUTOMOTIVE PRODUCTS (IDENTIFIED AS "XA" IN THE PART NUMBER) ARE NOT WARRANTED FOR USE IN THE DEPLOYMENT OF AIRBAGS OR FOR USE IN APPLICATIONS THAT AFFECT CONTROL OF A VEHICLE ("SAFETY APPLICATION") UNLESS THERE IS A SAFETY CONCEPT OR REDUNDANCY FEATURE CONSISTENT WITH THE ISO 26262 AUTOMOTIVE SAFETY STANDARD ("SAFETY DESIGN"). CUSTOMER SHALL, PRIOR TO USING OR DISTRIBUTING ANY SYSTEMS THAT INCORPORATE PRODUCTS, THOROUGHLY TEST SUCH SYSTEMS FOR SAFETY PURPOSES. USE OF PRODUCTS IN A SAFETY APPLICATION WITHOUT A SAFETY DESIGN IS FULLY AT THE RISK OF CUSTOMER, SUBJECT ONLY TO APPLICABLE LAWS AND REGULATIONS GOVERNING LIMITATIONS ON PRODUCT LIABILITY.

Copyright

© Copyright 2017-2023 Advanced Micro Devices, Inc. AMD, the AMD Arrow logo, Vitis, Vivado, and combinations thereof are trademarks of Advanced Micro Devices, Inc. Other product names used in this publication are for identification purposes only and may be trademarks of their respective companies.